THE DATA ENGINEER LAB

Day 20

Databricks Asset Bundles / Declarative Automation Bundles

CI/CD for Databricks - Deep Dive Notes

2026 terminology: Databricks Asset Bundles were officially renamed Declarative Automation Bundles in March 2026. The bundle CLI commands and the core model remain the same.

What problem are Bundles solving?

In a development workspace, it is easy to create a Job, pipeline, schedule, cluster configuration, permissions, and source code by clicking in the Databricks UI. The real challenge starts when the same project must move consistently through Dev, QA/Staging, and Production.

Approach	Meaning
Without a bundle	Manual resource creation, repeated configuration, drift between environments, difficult reviews, weak reproducibility.
With a bundle	Project source + Databricks resource definitions + deployment settings live together as code and can be validated and deployed repeatedly.
CI/CD layer	GitHub Actions, Azure DevOps, GitLab CI, Jenkins, etc. can automate the bundle commands.

The simplest mental model

```
Developer -> Git -> CI/CD pipeline -> Bundle validate/deploy -> Databricks workspace
```

Think of the bundle as the deployable definition of the Databricks project. Think of the CI/CD system as the automation engine that decides when and under what controls that deployment happens.

1. What exactly is a Declarative Automation Bundle?

Databricks describes a bundle as an end-to-end project definition that brings software-engineering practices such as source control, code review, testing, and CI/CD to data and AI projects. A bundle can package source files, metadata, resource definitions, infrastructure/workspace configuration, deployment targets, and other artifacts into a single deployable project.

- Source files: notebooks, Python files, SQL and other project code.
- Databricks resources: Jobs, Lakeflow pipelines, dashboards, apps, and other supported resources.
- Deployment configuration: workspace targets, variables, permissions, run identity, presets and modes.
- Artifacts and libraries: for example Python wheels, JARs and package dependencies where required.
- Environment-specific settings: Dev, Staging/QA and Production can use different target configuration.

Important distinction: A Bundle is broader than a Job definition. A Job can be one resource inside the Bundle.

2. Core project structure

```
my-data-project/
├── databricks.yml
├── resources/
│   ├── job.yml
│   └── pipeline.yml
├── src/
│   ├── bronze.py
│   ├── silver.py
│   └── gold.py
├── tests/
│   └── test_transformations.py
└── pyproject.toml
```

A bundle must contain exactly one databricks.yml at the root. That file is the main bundle configuration and can include additional YAML files, which is why larger projects usually keep resource definitions under a resources/ directory.

3. What does databricks.yml control?

- Bundle name and project identity.
- Included configuration files.
- Variables and artifacts.
- Deployment targets such as dev, qa, or prod.
- Workspace configuration and target-specific overrides.
- Permissions, run identity, Git settings, presets, and deployment mode.

4. A practical databricks.yml example

```
bundle:
  name: sales_etl

include:
  - resources/*.yml

targets:
  dev:
    mode: development
    workspace:
      host: https://<dev-workspace>

  prod:
    mode: production
    workspace:
      host: https://<prod-workspace>
    git:
      branch: main
```

This is intentionally simplified. The key concept is that the same project can define multiple targets while applying target-specific settings.

5. Development vs Production mode

Area	Development mode	Production mode
Purpose	Fast iteration and pre-production validation	Controlled deployment of validated deliverables
Schedules/triggers	Paused by default for deployed resources	Normal production behavior
Concurrent job runs	Enabled for faster iteration	Not automatically loosened for development convenience
Deployment lock	Disabled by default	Production-oriented controls apply
Git branch validation	Not the main production safeguard	Can validate deployment branch against configured branch
Identity	Often developer identity	Databricks recommends service principals for production deployments

Nuance: Deployment modes are optional convenience presets. Targets can exist without setting mode: development or mode: production.

6. Targets are the environment abstraction

A target answers: Where should this project deploy, and what should be different in this environment? Targets can point to different workspaces, identities, variables, paths, permissions, and resource overrides.

```
databricks bundle deploy -t dev
databricks bundle deploy -t prod
```

7. Bundle lifecycle and CLI workflow

1. Create or initialize the bundle project.
2. Develop source code and resource configuration locally or in the workspace.
3. Validate the configuration against Databricks resource schemas.
4. Deploy the bundle to a target workspace.
5. Run the deployed workflow resource when required.
6. Destroy the bundle deployment if it is no longer needed.

```
databricks bundle init
databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run -t dev <resource-key>
databricks bundle destroy -t dev
```

What validate, deploy and run actually mean

Command	Purpose
bundle validate	Checks bundle settings and resource definitions against the expected schemas and resolves configuration before deployment.
bundle deploy	Creates or updates the bundle-managed resources in the chosen target workspace.
bundle run	Triggers a deployed resource such as a Job or pipeline.
bundle plan	Previews deployment changes before applying them.
bundle deployment bind	Links a bundle-defined resource to an existing workspace resource so the bundle can manage it.

8. Existing resources can be onboarded

You do not always need to rebuild a Job or pipeline from scratch. Current Databricks tooling can generate configuration for existing resources and bind the bundle definition to an existing workspace resource. This is useful when a team wants to move from UI-managed resources to version-controlled deployments.

9. How CI/CD fits around Bundles

```
Developer
   |
Feature branch
   |
Pull Request -> Code Review -> Unit / Integration Tests
   |
Merge to main
   |
CI/CD Pipeline
   |
Databricks CLI: validate -> deploy
   |
Production Databricks workspace
```

Bundle vs CI/CD system

Component	Responsibility	Example
Bundle	Defines project resources, deployment targets, artifacts, variables and deployment behavior.	databricks.yml + resources/*.yml + src/
Git repository	Versions the bundle and enables collaboration/review.	GitHub / Azure Repos / GitLab
CI/CD platform	Runs checks and bundle commands when conditions are met.	GitHub Actions / Azure DevOps / Jenkins
Databricks workspace	Receives and runs the deployed resources.	Dev / QA / Production workspace

Interview-safe wording: Do not say "Bundles are the CI/CD server." A better statement is: Bundles make Databricks projects deployable as code, while a CI/CD platform automates validation and deployment.

10. Production pattern

- Keep bundle configuration and application code in Git.
- Use pull requests and automated tests before merge.
- Deploy development targets frequently for fast iteration.
- Use production mode and controlled branches for production deployments.
- Prefer a service principal or other team-owned deployment identity for production, as recommended by Databricks.
- Use permissions and a team-owned production root path rather than tying production state to an individual developer.
- Use variables/target overrides instead of duplicating the entire project for every environment.

11. Example: Medallion ETL project with Bundles

Suppose a team has Bronze, Silver and Gold notebooks plus a Lakeflow Job that runs them in sequence. The team needs the project in Dev and Production.

7. Develop bronze.py, silver.py and gold.py under src/.
8. Define the Lakeflow Job under resources/job.yml.
9. Use databricks.yml to include resource YAML and define dev/prod targets.
10. Validate locally before deployment.
11. Deploy to dev and test the Job.
12. Open a pull request and run automated tests.
13. After approval and merge, the CI/CD pipeline authenticates to Databricks and deploys the prod target.
14. The Production Job is now created/updated from the version-controlled bundle definition.

12. Common mistakes

- Treating YAML as the only thing that matters and forgetting source code, tests, artifacts and identities.
- Hard-coding environment-specific workspace paths or IDs everywhere instead of using targets and variables.
- Deploying Production using a personal user identity when a team-owned service principal is more appropriate.
- Manually changing bundle-managed resources in Production and creating configuration drift.
- Saying Bundle = CI/CD. It is one part of a CI/CD solution, not the pipeline engine itself.
- Duplicating full Dev and Prod configuration instead of sharing common configuration and overriding only what differs.

13. Interview Q&A

Q: What are Declarative Automation Bundles?

A: A way to define a Databricks project - source files, resource configuration and deployment settings - as a version-controlled deployable unit.

Q: What was the old name?

A: Databricks Asset Bundles. Databricks renamed them Declarative Automation Bundles in March 2026.

Q: What is databricks.yml?

A: The required root configuration file that defines the bundle and can include other configuration files.

Q: What is a target?

A: A named deployment environment definition such as dev, qa or prod with environment-specific workspace and configuration settings.

Q: Does a Bundle replace GitHub Actions or Azure DevOps?

A: No. Those systems automate CI/CD. The Bundle defines and performs the Databricks-specific validation/deployment workflow.

Q: How do you move the same project from Dev to Prod?

A: Keep a shared bundle project, define targets/overrides, validate it, and deploy the required target through a controlled CI/CD process.

Q: What identity should production deployments use?

A: Databricks recommends service principals for production deployments.

14. Official references

- **What are Declarative Automation Bundles?:** https://docs.databricks.com/aws/en/dev-tools/bundles
- **Develop Declarative Automation Bundles:** https://docs.databricks.com/aws/en/dev-tools/bundles/work-tasks
- **Deployment modes:** https://docs.databricks.com/aws/en/dev-tools/bundles/deployment-modes
- **Bundle configuration / databricks.yml:** https://docs.databricks.com/gcp/en/dev-tools/bundles/settings
- **Bundle configuration reference:** https://docs.databricks.com/aws/en/dev-tools/bundles/reference
- **Bundle CLI commands:** https://docs.databricks.com/aws/en/dev-tools/cli/bundle-commands
- **Bundle examples:** https://docs.databricks.com/aws/en/dev-tools/bundles/examples
- **Bundle release notes:** https://docs.databricks.com/aws/en/release-notes/dev-tools/bundles

Day 20 takeaway

Remember this flow: Code + Git + CI/CD + Declarative Automation Bundle + Databricks target = repeatable, reviewable and production-friendly Databricks deployments.